

- 2 — CRUD & DB Modelling
 - Table of Contents
 - 1. What this project covers
 - 2. The data model at a glance
 -  Detailed Architecture Diagrams
 - 1. Database Schema with Relationships
 - 2. 1:N Relationship (Department → Employees)
 - 3. 1:1 Relationship (Employee → Profile)
 - 4. N:N Relationship (Employees ↔ Projects)
 - 5. API Request Flow with Eager Loading
 - 6. CRUD Operations Flow
 - 7. Error Handling & Validation Flow
 - 3. Foreign keys & cascade rules
 - 4. Modelling cardinalities in Drizzle
 - 4.1 1:N — departments ⇔ employees
 - 4.2 1:1 — employees ⇔ employee_profiles
 - 4.3 N:N — employees ⇔ projects via employee_projects
 - 5. The Drizzle relational query API
 - 6. Eager loading vs. N+1
 - 7. REST design for related resources
 - 8. Validation with zod
 - 9. Pagination, sorting, filtering
 - 10. Mapping database errors to HTTP responses
 - 11. Migrations & seed strategy
 - 12. Project structure
 - 13. Daily workflow
 - 14. Troubleshooting
 - See also

2 — CRUD & DB Modelling

A relational REST API built with **NestJS 11**, **Drizzle ORM 0.45**, **PostgreSQL 16**, and **zod 4**, fully containerized. Continues from [1.project-setup](#) and adds the three relationship cardinalities every backend dev needs to be fluent in: **1:N**, **1:1**, **N:N**.

 For the practical, command-by-command build instructions, see [step-by-step.md](#). This README is the **conceptual reference** — every section explains

Table of Contents

1. [What this project covers](#)
 2. [The data model at a glance](#)
 3. [Foreign keys & cascade rules](#)
 4. [Modelling cardinalities in Drizzle](#)
 - 1:N — [departments](#) ⇔ [employees](#)
 - 1:1 — [employees](#) ⇔ [employee_profiles](#)
 - N:N — [employees](#) ⇔ [projects](#) [via](#) [employee_projects](#)
 5. [The Drizzle relational query API](#)
 6. [Eager loading vs. N+1](#)
 7. [REST design for related resources](#)
 8. [Validation with zod](#)
 9. [Pagination, sorting, filtering](#)
 10. [Mapping database errors to HTTP responses](#)
 11. [Migrations & seed strategy](#)
 12. [Project structure](#)
 13. [Daily workflow](#)
 14. [Troubleshooting](#)
-

1. What this project covers

Topic	Where it lives	Why it matters
Foreign keys, cascade rules	src/**/schema/*.schema.ts	DB-level integrity is your last line of defense.
1:N modelling	departments ⇔ employees	The most common shape in business data.
1:1 modelling	employees ⇔ employee_profiles	How to split optional/heavy columns into a sibling table.

Topic	Where it lives	Why it matters
N:N modelling + junction with metadata	<code>employees ↔ projects</code> via <code>employee_projects</code>	Real-world links carry their own attributes (<code>role</code> , <code>allocation</code>).
Drizzle relational queries (<code>db.query.*</code>)	<code>src/database/relations.ts</code>	Type-safe, no string SQL, no decorators.
Eager loading via <code>?include=...</code>	<code>*.controller.ts</code> + <code>*.service.ts</code>	Solves N+1 without leaking the ORM to clients.
zod validation + cross-field rules	<code>src/**/dto/*.ts</code>	Runtime safety + free TS types.
Pagination / sorting / filtering	<code>list-*.dto.ts</code> , <code>*.service.ts</code>	Standard contract for any list endpoint.
Translating DB errors to HTTP	<code>*.service.ts</code>	No more <code>500</code> on a duplicate email.
Seed orchestration with FKs	<code>src/database/seed/index.ts</code>	Insert order matters when relationships exist.

2. The data model at a glance

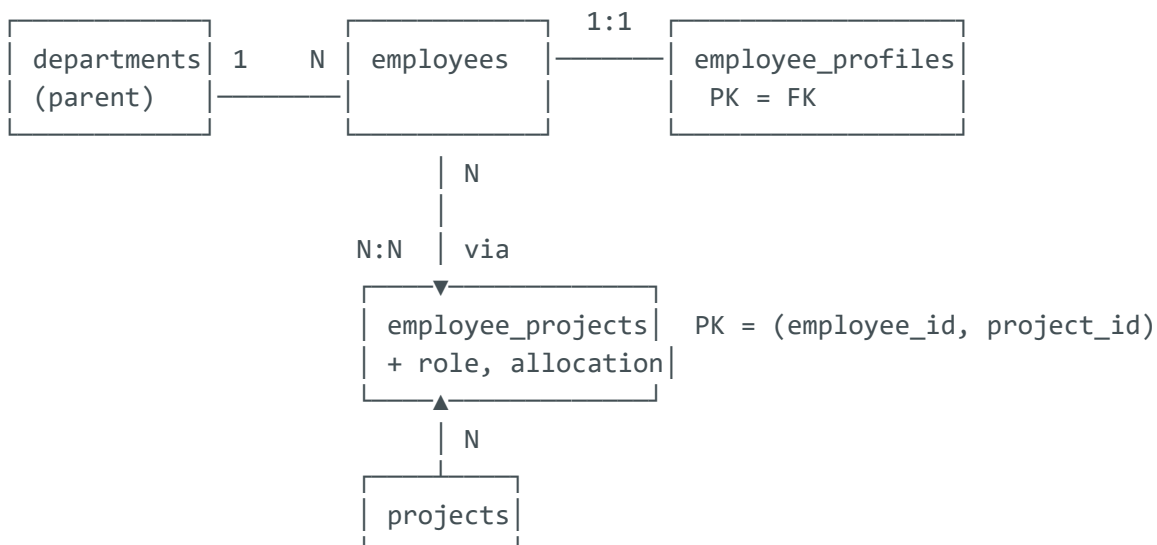
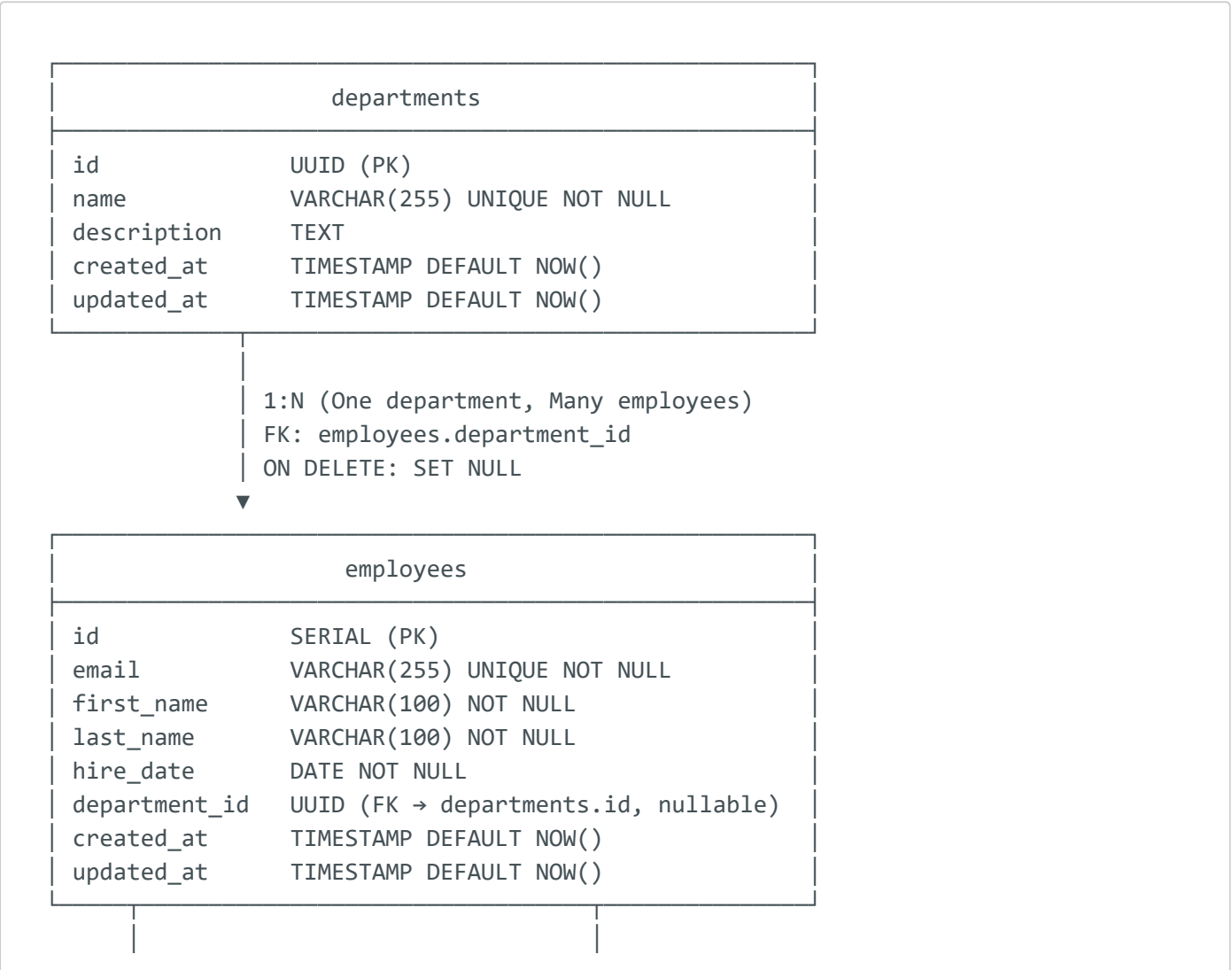
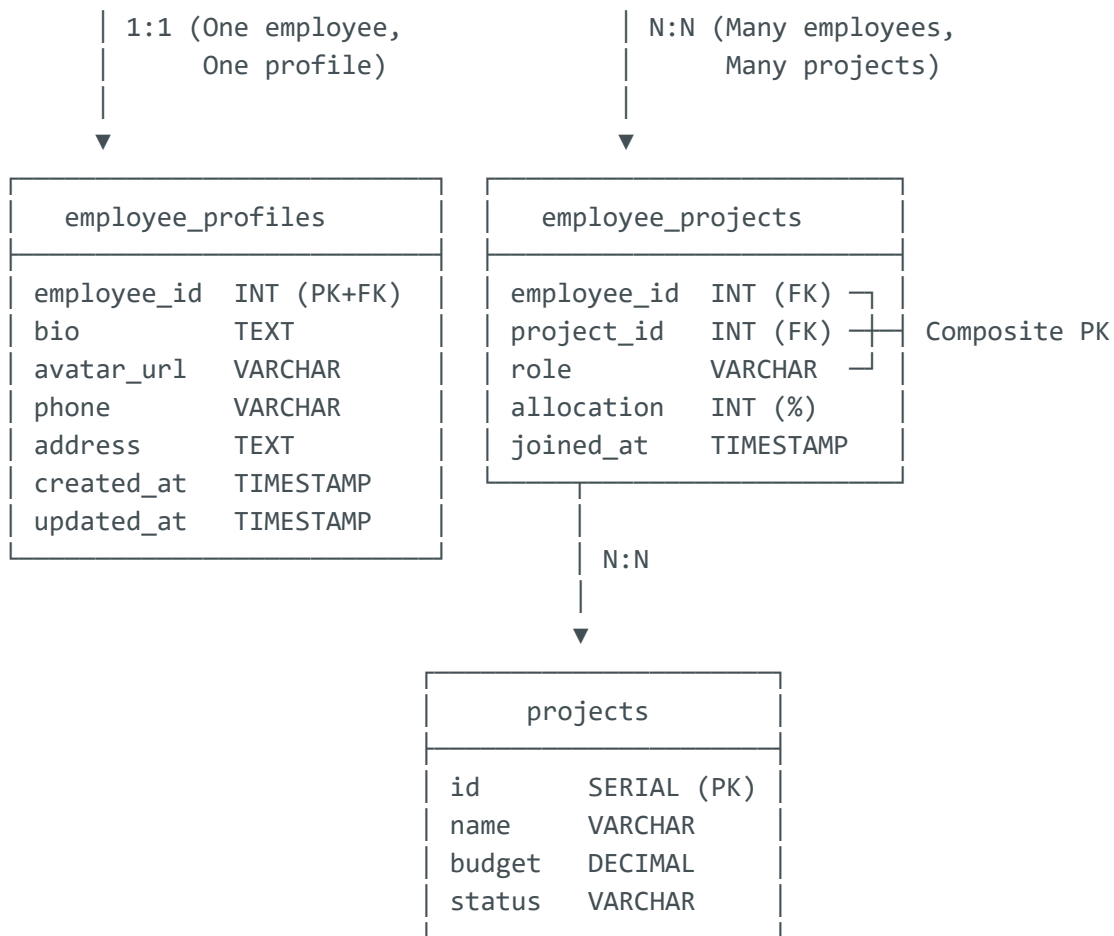


Table	Purpose	Key columns
departments	Parent of a 1:N	id (PK, uuid), name (unique)
employees	Central entity	id (PK), email (unique), department_id (FK, nullable)
employee_profiles	1:1 sibling, optional	employee_id (PK + FK)
projects	One side of an N:N	id (PK), name (unique)
employee_projects	Junction with metadata	composite PK (employee_id, project_id), role, allocation

Detailed Architecture Diagrams

1. Database Schema with Relationships



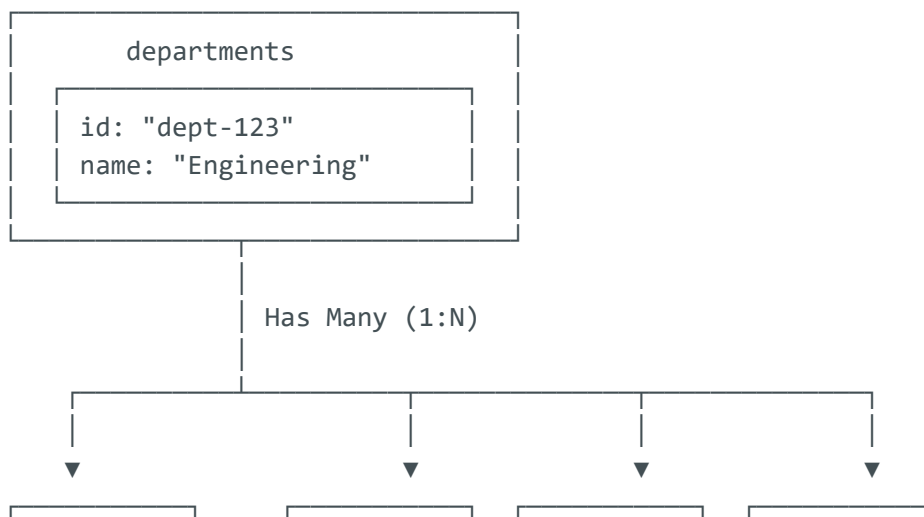


Cascade Rules:

- Department deleted → employee.department_id = NULL (SET NULL)
- Employee deleted → profile deleted (CASCADE)
- Employee deleted → assignments deleted (CASCADE)
- Project deleted → assignments deleted (CASCADE)

2. 1:N Relationship (Department → Employees)

One Department, Many Employees



Employee #1	Employee #2	Employee #3	Employee #4
id: 101 dept_id: "dept-123" name: "Alice"	id: 102 dept_id: "dept-123" name: "Bob"	id: 103 dept_id: "dept-123" name: "Carol"	id: 104 dept_id: "dept-123" name: "Dave"

Query Examples:

1. Get department with all employees (Eager Loading):

```
GET /departments/dept-123?include=employees
```

```
SELECT departments.*, employees.*
FROM departments
LEFT JOIN employees ON employees.department_id = departments.id
WHERE departments.id = 'dept-123'
```

2. Get employee with department (Reverse):

```
GET /employees/101?include=department
```

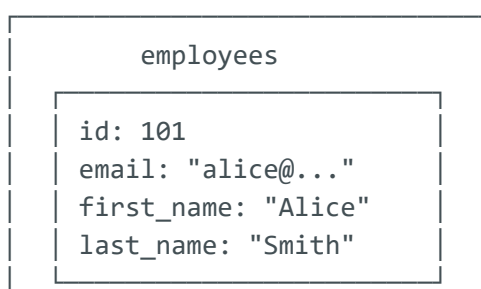
```
SELECT employees.*, departments.*
FROM employees
LEFT JOIN departments ON departments.id = employees.department_id
WHERE employees.id = 101
```

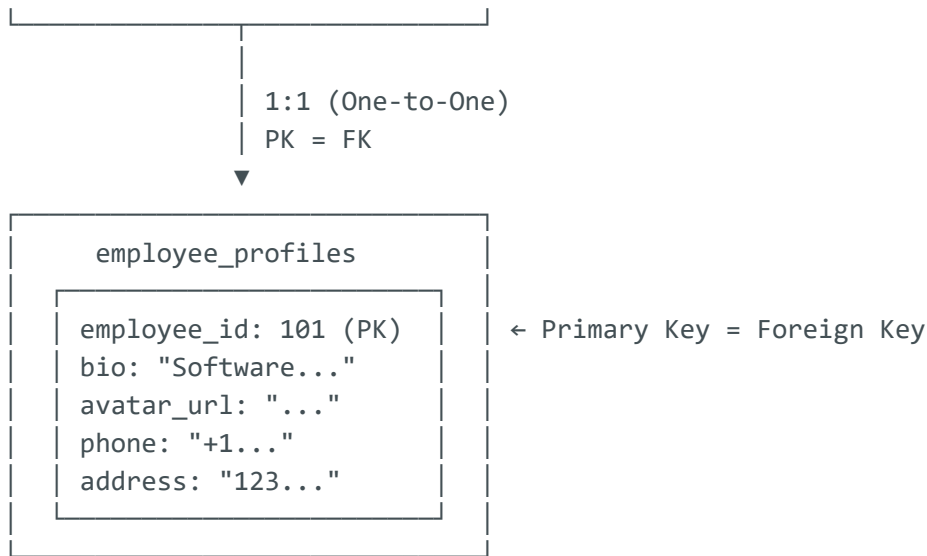
Cascade Behavior:

```
DELETE FROM departments WHERE id = 'dept-123'
↓
ON DELETE SET NULL
↓
UPDATE employees
SET department_id = NULL
WHERE department_id = 'dept-123'
↓
Result: Employees 101-104 now have department_id = NULL
```

3. 1:1 Relationship (Employee → Profile)

One Employee, One Profile (Optional)





Why 1:1?

- ✓ Optional data (not all employees have profiles)
- ✓ Large/heavy columns (bio, address, avatar)
- ✓ Different access patterns (main data vs extended)
- ✓ Separation of concerns (core vs optional)

Implementation:

- employee_id is BOTH Primary Key AND Foreign Key
- Profile can only exist if employee exists
- One employee can have at most one profile

Query Examples:

1. Get employee with profile:

GET /employees/101?include=profile

```
SELECT employees.*, employee_profiles.*
FROM employees
LEFT JOIN employee_profiles
  ON employee_profiles.employee_id = employees.id
WHERE employees.id = 101
```

Cascade Behavior:

```
DELETE FROM employees WHERE id = 101
```

↓

```
ON DELETE CASCADE
```

↓

```
DELETE FROM employee_profiles WHERE employee_id = 101
```

↓

Result: Profile automatically deleted (can't exist without employee)

4. N:N Relationship (Employees ↔ Projects)

Many Employees, Many Projects (Junction Table)

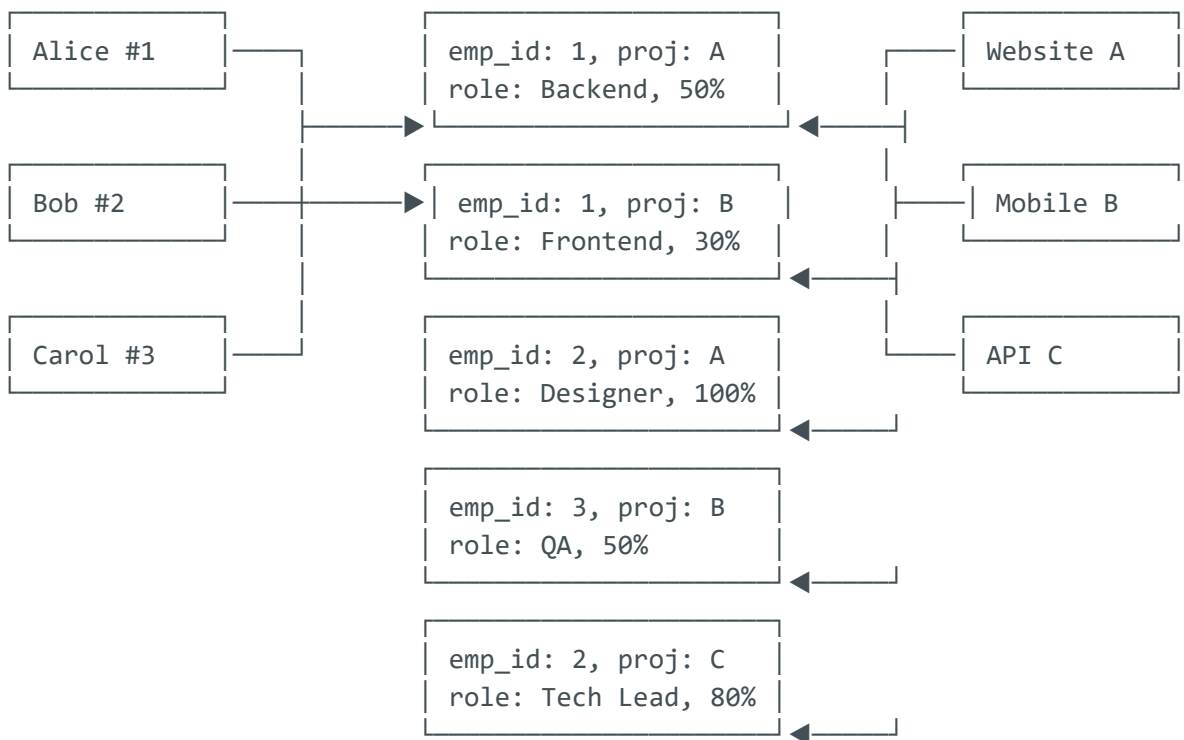


Full Example with Multiple Relationships:

Employees:

Junction (Assignments):

Projects:



Junction Table Schema:

employee_projects		
employee_id	INT (FK → employees.id)	} Composite PK
project_id	INT (FK → projects.id)	
role	VARCHAR(100)	
allocation	INT (percentage)	
joined_at	TIMESTAMP	

Benefits of Junction with Metadata:

- ✓ Stores attributes of the relationship (role, allocation)

- ✓ Flexible querying (find all projects for an employee)
- ✓ Prevents duplicates (composite PK)
- ✓ Clean cascade on delete

Query Examples:

1. Get all projects for Alice (employee #1):

GET /employees/1/projects

```
SELECT projects.*, employee_projects.role, employee_projects.allocation
FROM projects
JOIN employee_projects ON employee_projects.project_id = projects.id
WHERE employee_projects.employee_id = 1
```

Result: Website (Backend, 50%), Mobile (Frontend, 30%)

2. Get all employees on Website (project A):

GET /projects/A/employees

```
SELECT employees.*, employee_projects.role, employee_projects.allocation
FROM employees
JOIN employee_projects ON employee_projects.employee_id = employees.id
WHERE employee_projects.project_id = 'A'
```

Result: Alice (Backend, 50%), Bob (Designer, 100%)

5. API Request Flow with Eager Loading

Request: GET /employees/101?include=department,profile,projects



```
    employeeProjects: {  
      with: { project: true }  
    }  
  }  
})
```

Single SQL query with JOINS

PostgreSQL

```
SELECT  
  employees.*,  
  departments.*,  
  employee_profiles.*,  
  employee_projects.*,  
  projects.*  
FROM employees  
LEFT JOIN departments  
  ON departments.id = employees.dept_id  
LEFT JOIN employee_profiles  
  ON profiles.employee_id = employees.id  
LEFT JOIN employee_projects  
  ON ep.employee_id = employees.id  
LEFT JOIN projects  
  ON projects.id = ep.project_id  
WHERE employees.id = 101
```

Returns joined data in single round-trip

Drizzle ORM (Auto-maps to objects)

```
{  
  id: 101,  
  email: "alice@example.com",  
  department: {  
    id: "dept-123",  
    name: "Engineering"  
  },  
  profile: {  
    bio: "Software engineer...",  
    phone: "+1-555-1234"  
  },  
  employeeProjects: [  
    {  
      role: "Backend Dev",  
      allocation: 50,  
      project: {  
        name: "Website Redesign"  
      }  
    }  
  ]  
}
```

Return formatted response

Client

← 200 OK with full nested data

Performance:

- ✓ Single SQL query (no N+1 problem)
- ✓ Database does the joining (efficient)
- ✓ Type-safe at compile time (Drizzle)
- ✓ Flexible includes (client controls depth)

Without Eager Loading (N+1 Problem):

1 query: Get employee

1 query: Get department

1 query: Get profile

N queries: Get each project assignment

= (3 + N) queries! ✗ Slow!

With Eager Loading:

1 query: Get everything at once ✓ Fast!

6. CRUD Operations Flow

CREATE (POST):

Client → POST /employees

```
{
  "email": "alice@example.com",
  "firstName": "Alice",
  "departmentId": "dept-123"
}
```

↓

Controller: Validate with Zod

↓

Service: Insert into PostgreSQL

↓

```
INSERT INTO employees (email, first_name, department_id)
VALUES ('alice@...', 'Alice', 'dept-123')
RETURNING *
```

↓

Response: 201 Created

```
{id: 101, email: "alice@...", ...}
```

READ (GET):

Client → GET /employees?page=1&limit=10&sort=firstName

↓

Controller: Parse query params

↓

Service: Build paginated query

↓

```
SELECT * FROM employees
ORDER BY first_name ASC
LIMIT 10 OFFSET 0
↓
Response: 200 OK
{
  data: [{id: 101, ...}, ...],
  meta: {page: 1, total: 50}
}
```

UPDATE (PUT/PATCH):

```
Client → PATCH /employees/101
{
  "firstName": "Alicia",
  "departmentId": "dept-456"
}
↓
Controller: Validate partial update
↓
Service: Check if exists, then update
↓
UPDATE employees
SET first_name = 'Alicia',
    department_id = 'dept-456',
    updated_at = NOW()
WHERE id = 101
RETURNING *
↓
Response: 200 OK
{id: 101, firstName: "Alicia", ...}
```

DELETE (DELETE):

```
Client → DELETE /employees/101
↓
Controller: Validate ID
↓
Service: Check if exists
↓
DELETE FROM employees WHERE id = 101
↓
CASCADE: Also deletes:
- employee_profiles (101)
- employee_projects (employee_id = 101)
↓
Response: 204 No Content
```

7. Error Handling & Validation Flow

Invalid Request:

Client → POST /employees

```
{  
  "email": "not-an-email", ← Invalid  
  "firstName": ""           ← Empty  
}
```

↓

Zod Validation

email: z.string().email()
→ ❌ Invalid email format

firstName: z.string().min(1)
→ ❌ String must contain at
least 1 character(s)

↓

400 Bad Request

```
{  
  "statusCode": 400,  
  "message": "Validation failed",  
  "errors": {  
    "email": "Invalid email",  
    "firstName": "Required"  
  }  
}
```

Database Constraint Violation:

Client → POST /employees

```
{  
  "email": "alice@example.com" ← Already exists  
}
```

↓

Zod validation passes ✓

↓

Try INSERT INTO employees...

↓

PostgreSQL Error

Code: 23505

Message: duplicate key value
violates unique constraint

"employees_email_unique"

↓

Error Handler

(PostgresErrorService)

```
Detect: error.code === '23505'  
Field: error.detail contains  
"email"
```



```
409 Conflict  
  
{  
  "statusCode": 409,  
  "message": "Email already  
             exists",  
  "field": "email"  
}
```

3. Foreign keys & cascade rules

Every FK in this project picks an explicit **onDelete** policy. The choice changes how the system behaves under deletion:

Relationship	onDelete	Effect
employees.department_id → departments.id	set null	Deleting a department keeps its employees; their FK is cleared.
employee_profiles.employee_id → employees.id	cascade	Deleting an employee removes the profile (it has no meaning alone).
employee_projects.employee_id → employees.id	cascade	Removing an employee removes their assignments.
employee_projects.project_id → projects.id	cascade	Removing a project removes all its assignments.

The third option — **restrict** — refuses the delete if children exist. Pick it when losing related data is worse than failing the request.

Always model the cascade explicitly. The default in many ORMs is "no action," which surfaces as a confusing FK violation at delete time.

4. Modelling cardinalities in Drizzle

4.1 1:N — departments ⇔ employees

The FK lives on the **child** (the "many" side). The parent table has no extra columns.

```
// child
departmentId: uuid('department_id')
  .references(() => departments.id, { onDelete: 'set null' })),
```

A nullable FK ("optional parent") models "an employee may belong to a department." A `notNull()` FK would model "an employee **must** belong to a department."

A secondary index on the FK column (`employees_department_id_idx`) is almost always worth it — listing employees by department becomes an indexed lookup instead of a scan.

4.2 1:1 — employees ⇔ employee_profiles

There are two common ways to enforce 1:1:

Pattern	Pros	Cons
PK = FK (<i>used here</i>)	Uniqueness for free; one row per parent guaranteed by the PK.	Child can't exist before the parent.
Separate PK + <code>unique()</code> on the FK	Child can have its own surrogate ID.	Extra column, extra index.

```
employeeId: uuid('employee_id')
  .primaryKey() // ← unique by being PK
  .references(() => employees.id, { onDelete: 'cascade' })),
```

When to split a 1:1 out at all? Two main reasons:

- **Optional, heavy columns** (long bios, avatars) you don't want to load on every employee query.
- **Sparse data** that changes ownership (auth providers, integrations).

4.3 N:N — employees ⇔ projects via employee_projects

A junction is its own table. Two design rules to internalize:

1. **Composite primary key on the FK pair.** This is what prevents duplicate links — no extra `unique()` constraint or "if exists" check needed in the service. Postgres returns `23505` if you try.
2. **The junction is the natural home for per-link attributes** (`role`, `allocation`, `assigned_at`). The moment the relationship has its own facts, it stops being "implicit" and becomes a first-class entity with its own routes.

```
primaryKey({ columns: [t.employeeId, t.projectId] })
```

5. The Drizzle relational query API

Foreign keys give you SQL integrity; `relations()` gives you type-safe traversal:

```
db.query.employees.findMany({
  with: {
    department: true,           // 1:N parent → one()
    profile: true,              // 1:1 → one()
    projects: { with: { project: true } }, // N:N → nest through junction
  },
});
```

Two rules to remember:

- **one()** on both sides of a 1:1 — Drizzle treats it as a 1:N where the FK is unique.
- **N:N requires nesting** — `projects: many(employeeProjects)` only reaches the junction. To get the other parent, you nest a `with` inside the junction.

Relations are **declarations**, not queries. They're inert until you use them in a **with**. Defining them does not change the SQL of plain **.select()** calls.

6. Eager loading vs. N+1

The naive way to fetch employees with their department is two queries per row — the textbook **N+1**. The Drizzle **with** clause issues a single query (or a small fixed number) regardless of result size.

This project exposes eager loading through a controlled query string:

```
GET /employees?include=department,profile,projects
GET /employees/:id?include=profile
GET /departments?includeEmployees=true
GET /projects/:id?includeMembers=true
```

Two security/perf rules:

- 1. **Allow-list the includable relations** — never pipe raw **?include=** into **with**. An attacker should not be able to traverse arbitrary tables.
- 2. **Cap the pageSize** when an include is active, or paginate the children. Eager-loading 10,000 projects per employee is a footgun.

7. REST design for related resources

Pattern	Used for	Routes
Top-level resource	Independent entities	/employees, /departments, /projects
Sub-resource (1:1)	Owned by one parent	GET/PUT /employees/:id/profile
Sub-resource (collection)	Lists scoped to a parent	GET /employees/:id/projects, GET /projects/:id/members

Pattern	Used for	Routes
Junction resource	When the link itself is a thing	<code>POST /projects/:id/members</code> , <code>DELETE /projects/:id/members/:employeeId</code>

Conventions used in this project:

- **PUT for upserts** on 1:1 sub-resources — the URL implies "the profile of employee X," so PUT semantics (create-or-replace) match.
- **PATCH for partial updates** on top-level resources — the body contains only changed fields.
- **DELETE returns 204 No Content** — no body.
- **Composite-key sub-resources** put the parent's ID in the path and the child's discriminator after it: `/projects/:id/members/:employeeId`.

8. Validation with zod

Each DTO is a zod schema; the inferred TypeScript type is exported alongside it so the controller, service, and tests share one source of truth:

```
export const createProjectSchema = z.object({
  name: z.string().min(1).max(150),
  startDate: z.string().date().optional(),
  endDate: z.string().date().optional(),
}).strict() // unknown keys → 400
  .refine(
    (v) => !v.startDate || !v.endDate || v.startDate <= v.endDate,
    { message: 'endDate must be on or after startDate', path: ['endDate'] },
  );

export type CreateProjectDto = z.infer<typeof createProjectSchema>;
```

Three patterns used throughout:

- **.strict()** so unknown fields are rejected. Stops typos from being silently dropped.
- **.refine()** for cross-field rules (date order, conditional presence). Single-field rules belong on the field itself.
- **createSchema.partial()** for the corresponding update DTO, so they never drift apart.

A tiny `ZodValidationPipe` + `@ZodBody` / `@ZodQuery` / `@ZodParam` decorators convert zod failures into Nest's `BadRequestException` with the issue list attached.

9. Pagination, sorting, filtering

Every list endpoint accepts the same query contract:

Param	Default	Notes
<code>page</code>	<code>1</code>	1-based
<code>pageSize</code>	<code>20</code>	Max <code>100</code> (rejected at the DTO)
<code>sortBy</code>	<code>createdAt</code>	Allow-list in the zod enum
<code>sortOrder</code>	<code>desc</code>	<code>asc</code> <code>desc</code>
<code>search</code>	—	Case-insensitive across selected text columns
Resource-specific filters	—	e.g. <code>departmentId</code> , <code>position</code> , <code>isActive</code>
<code>include</code> / <code>includeXxx</code>	—	Eager-load relations

Response envelope:

```
{
  "data": [/* rows */],
  "meta": { "page": 1, "pageSize": 20, "total": 42, "totalPages": 3 }
}
```

Implementation note: run the rows query and the `count(*)` in **parallel** with `Promise.all` — they're independent and the count is otherwise pure latency.

The `sortBy` allow-list (a `z.enum`) is the critical defense against arbitrary `ORDER BY` injection from the URL.

10. Mapping database errors to HTTP responses

A **500 Internal Server Error** on a duplicate email is a bug, not behavior. Catch the Postgres error and translate:

```
const PG_UNIQUE_VIOLATION = '23505';
const PG_FK_VIOLATION     = '23503';
```

Drizzle wraps the underlying **pg** error on **.cause**, so check both layers:

```
private isUniqueViolation(err: unknown): boolean {
  return hasCode(err, PG_UNIQUE_VIOLATION)
    || hasCode((err as any)?.cause, PG_UNIQUE_VIOLATION);
}
```

Standard mappings used in this project:

Postgres code	Meaning	HTTP status
23505	unique violation	409 Conflict
23503	FK violation	409 Conflict (or 400 if it's a client-supplied bad ID)
23502	not-null violation	400 Bad Request (validation should catch this first)
22P02	invalid text representation (e.g. bad UUID)	400 Bad Request

Anything not explicitly mapped becomes a real **500** — that way unexpected errors stay loud.

11. Migrations & seed strategy

Migrations live in `src/database/migrations/` and are produced by `drizzle-kit generate` from the schema files. They are **the** source of truth for the deployed shape — never edit a migration after it's been applied to a shared environment; create a new one.

```
pnpm db:generate      # diff schema vs. last migration → new SQL file
pnpm db:migrate       # apply pending migrations
```

Seeding is for predictable demo / dev data. The order matters because of FKs:

```
departments → employees → employee_profiles → projects → employee_projects
```

Two modes:

- `pnpm db:seed` — **additive**, idempotent (skips rows that conflict on unique columns).
- `pnpm db:seed:reset` — `TRUNCATE ... RESTART IDENTITY CASCADE` first, then reseed.

Tests should never run against the dev seed; spin up a throwaway DB or a transaction-scoped fixture instead.

12. Project structure

```
2.CRUD & DB Modelling/
├─ docker/postgres/init-databases.sh      # creates dev + prod DBs on first
start
├─ scripts/docker.sh                     # compose wrapper:
up/down/logs/psql/nuke
├─ src/
│   └─ common/
│       ├── decorators/zod.decorators.ts  # @ZodBody / @ZodQuery / @ZodParam
│       └─ pipes/zod-validation.pipe.ts
│   └─ config/index.ts
│   └─ database/
│       ├── migrations/                  # generated SQL – do not edit manually
│       ├── seed/{departments,employees,projects}.seed.ts + index.ts
│       ├── database.module.ts           # global; provides DRIZZLE token
│       ├── database.constants.ts
│       └─ relations.ts                  # the relational query API
```

```
| | └─ schema.ts # barrel re-export
| └─ departments/{controller,service,module}.ts + dto/ + schema/
| └─ employees/ {controller,service,module}.ts + dto/ + schema/
| └─ projects/ {controller,service,module}.ts + dto/ + schema/
| └─ app.module.ts
| └─ main.ts
└─ .env.{development,production,example}
└─ app.http
└─ docker-compose.yml
└─ dockerfile
└─ drizzle.config.ts
└─ package.json
└─ README.md ← you are here (concepts)
└─ step-by-step.md ← practical build guide
```

13. Daily workflow

```
# Bring up api + postgres
./scripts/docker.sh up

# Tail logs
./scripts/docker.sh logs

# psql against the active env
./scripts/docker.sh psql

# Rebuild after Dockerfile / dep changes
./scripts/docker.sh rebuild

# DB only, then run the API on the host with hot reload
./scripts/docker.sh db:up
pnpm start:dev

# Schema changes
pnpm db:generate && pnpm db:migrate

# Reset + reseed
./scripts/docker.sh nuke && ./scripts/docker.sh up && pnpm db:migrate && pnpm
db:seed
```

Test endpoints from VS Code: open [app.http](#), hit **Send Request** above any block.

14. Troubleshooting

Symptom	Likely cause	Fix
500 Internal Server Error on GET /employees	DB has no schema (fresh volume)	<code>pnpm db:migrate</code> (then <code>pnpm db:seed</code>)
getaddrinfo EAI_AGAIN postgres	API container not on the compose network	<code>docker compose --env-file .env.development up -d --force-recreate postgres</code>
Bind for 0.0.0.0:5432 failed: port is already allocated	Sibling project's postgres holds the port	Stop the other stack, or set <code>DB_PORT_HOST=5433</code> in <code>.env.development</code>
database "..." does not exist	<code>init-databases.sh</code> only runs on a fresh volume	<code>./scripts/docker.sh nuke && ./scripts/docker.sh up</code>
42P01: relation "employees" does not exist	Migrations not applied	<code>pnpm db:migrate</code>
409 Conflict on every POST	Composite-PK / unique violation triggered by stale seed data	<code>pnpm db:seed:reset</code>
pgAdmin / GUI: temporary failure in name resolution	Used <code>postgres</code> as host from your machine	Use <code>localhost</code> (the published port)

See also

- [step-by-step.md](#) — practical, command-by-command build guide.
- [app.http](#) — runnable request collection covering every endpoint and error case.
- [drizzle-orm docs — relations](#)
- [PostgreSQL error codes](#)